TF4015 – Sistem Kontrol Diskrit

Praktikum

Modul 1

Identifikasi Sistem Motor DC

Tim Pengajar TF4015

Program Studi Teknik Fisika
Fakultas Teknologi Industri
Institut Teknologi Bandung

Semester I 2026/2027

Daftar Isi

1 Pendahuluan **1**
1.1 Latar Belakang . . . 1
1.2 Tujuan Praktikum . . . 1
1.3 Peralatan dan Perangkat Lunak . . . 2
1.4 Persiapan Praktikum . . . 2

2 Teori Dasar **2**
2.1 Karakteristik Statik dan Ketidaklinieran . . . 2
2.2 Penguatan Driver Motor dan Saturasi . . . 3
2.3 Kompensasi *Deadzone* . . . 4
2.4 Model Orde Satu dengan Waktu Tunda . . . 4
2.5 Identifikasi Parameter dengan Metode Dua Titik . . . 4
2.6 Pemilihan Waktu Cacah . . . 5
2.7 Model Diskrit . . . 6

3 Keselamatan Kerja **6**

4 Prosedur Praktikum **6**
4.1 Percobaan 1: Karakteristik Sensor . . . 7
4.2 Percobaan 2: Penyetelan Penguatan Driver Motor . . . 7
4.3 Percobaan 3: Karakteristik Statik Tanpa Kompensasi . . . 8
4.4 Percobaan 4: Kompensasi *Deadzone* . . . 9
4.5 Percobaan 5: Respons Step . . . 9
4.6 Percobaan 6: Identifikasi Model, Waktu Cacah, dan Model Diskrit . . . 10

5 Laporan dan Analisis **11**
5.1 Ketentuan Laporan . . . 11
5.2 Analisis . . . 11

6 Tugas Tambahan **13**

A Kode Program Pendukung **14**

1 Pendahuluan

1.1 Latar Belakang

Pengendalian kecepatan putar motor merupakan kasus inti pada berbagai aplikasi, seperti elevator, ban berjalan, dan pengaturan laju aliran fluida melalui pompa. Sebelum sebuah sistem dikendalikan secara kalang tertutup, perancang perlu mengetahui hubungan antara masukan dan keluaran sistem tersebut. Hubungan ini dapat diperoleh secara analitik dari hukum-hukum fisika, atau secara eksperimental dengan memberikan masukan tertentu lalu mengamati keluarannya. Pendekatan kedua disebut **identifikasi sistem**.

Pada Modul 1, praktikan belum melakukan pengontrolan. Seluruh percobaan dilakukan secara **kalang terbuka**. Praktikan terlebih dahulu mengamati karakteristik sensor, kemudian menentukan karakteristik statik motor, yaitu hubungan antara masukan dan kecepatan tunak. Dari karakteristik ini akan terlihat ketidaklinieran, seperti *deadzone* dan saturasi, yang perlu diperbaiki dengan kompensasi sebelum sistem dikontrol.

Setelah dikompensasi, dinamika motor diidentifikasi dari respons step menggunakan model **orde satu dengan waktu tunda** (*first order plus dead time*, FOPDT). Model ini kemudian digunakan untuk menentukan waktu cacah dan diturunkan menjadi model diskrit. Model kontinu dan model diskrit hasil Modul 1 menjadi dasar perancangan pengontrol kecepatan (Modul 2), kontrol posisi (Modul 3), dan *pole placement* (Modul 4). Oleh karena itu, kualitas identifikasi pada modul ini akan sangat memengaruhi keberhasilan modul-modul berikutnya.

1.2 Tujuan Praktikum

Setelah menyelesaikan Modul 1, praktikan diharapkan mampu:

1. Menganalisis karakteristik sensor posisi dan kecepatan, serta pengaruh metode akuisisi terhadap kualitas data dan waktu proses.
2. Menyetel penguatan driver motor serta menentukan karakteristik statik dan spesifikasi kit praktikum.
3. Merancang dan menerapkan fungsi kompensasi untuk mengurangi *deadzone*.
4. Mengidentifikasi parameter model FOPDT dari respons step sistem kecepatan motor DC.
5. Menentukan waktu cacah, menurunkan model diskrit, dan memvalidasi model terhadap data percobaan.

1.3 Peralatan dan Perangkat Lunak

Tabel 1: Daftar peralatan praktikum Modul 1

No	Nama Peralatan	Keterangan
1	Kit praktikum SKD (kit yang sama dengan Modul 0)	Disediakan asisten
2	Obeng kecil untuk trimpot	Disediakan asisten
3	Laptop dengan Arduino IDE dan *library* Adafruit MCP4725	Milik praktikan
4	MATLAB (desktop atau MATLAB Online)	Milik praktikan
5	Perangkat lunak *spreadsheet*	Milik praktikan
6	Program praktikum: `SKD23.ino`, `6_main_statik`, `7_main_step`	Unduh dari repositori
7	Skrip MATLAB `identifikasi_fopdt.m`	Unduh dari repositori
8	Data dan laporan Modul 0 (terutama waktu proses T_{proses})	Hasil praktikum sebelumnya

1.4 Persiapan Praktikum

Sebelum praktikum, praktikan perlu:

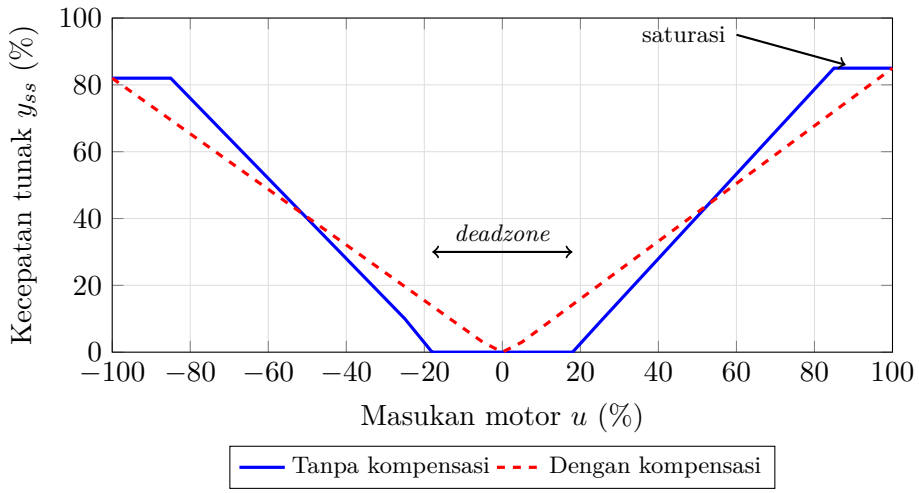
1. Menyelesaikan Modul 0 dan membawa hasil pengukuran waktu proses (Tabel perbandingan periode `loop()` pada Modul 0).
2. Membaca Bagian 2 dan memahami program `6_main_statik` serta `7_main_step` (Lampiran A).
3. Membaca skrip `identifikasi_fopdt.m` dan mencobanya pada MATLAB, misalnya dengan data buatan sendiri.
4. Menyiapkan templat *spreadsheet* untuk tabel dan grafik pada modul ini.
5. Membagi tugas dalam kelompok: pengoperasian kit, pencatatan data, dan pengolahan data dapat dilakukan secara paralel.

2 Teori Dasar

2.1 Karakteristik Statik dan Ketidaklinieran

Karakteristik statik adalah hubungan antara masukan u dan keluaran tunak y_{ss} suatu sistem, $y_{ss} = f(u)$. Pada sistem ideal, hubungan ini linier. Pada kit praktikum, hubungan masukan motor (persen) dan kecepatan terukur (persen) memiliki beberapa ketidaklinieran, seperti diilustrasikan pada Gambar 1:

- ***Deadzone***: pada masukan kecil, motor belum berputar atau putarannya belum terbaca oleh sensor sehingga keluaran bernilai nol.
- **Saturasi**: pada masukan besar, kecepatan tidak bertambah lagi meskipun masukan dinaikkan.
- **Asimetri**: karakteristik untuk masukan positif dan negatif dapat berbeda.



Gambar 1: Ilustrasi karakteristik statik motor dengan *deadzone*, saturasi, dan efek kompensasi (bukan data sebenarnya). Bacaan tachometer selalu positif untuk kedua arah putar.

Beberapa istilah spesifikasi yang digunakan pada modul ini:

- **Masukan minimum putar motor**: masukan terkecil yang membuat motor mulai berputar. Putaran dapat dideteksi dari perubahan bacaan potensiometer walaupun tachometer masih membaca nol.
- **Masukan minimum terdeteksi**: masukan terkecil yang menghasilkan bacaan kecepatan tidak nol.
- **Masukan minimum putaran maksimal**: masukan terkecil yang sudah menghasilkan kecepatan maksimum (awal daerah saturasi).

Karakteristik statik dapat didekati dengan beberapa **daerah linier**, yaitu potongan garis lurus $y_{ss} \approx m_i u + c_i$ pada rentang masukan tertentu. Pembagian daerah minimal harus memperlihatkan daerah *deadzone*, daerah naik, dan daerah saturasi. Kualitas pendekatan setiap daerah dapat dinilai dengan koefisien determinasi R^2. Pembagian yang terlalu banyak (lebih dari lima daerah) biasanya tidak lagi bermakna.

2.2 Penguatan Driver Motor dan Saturasi

Dari Modul 0, tegangan motor kurang lebih sebesar

$$V_m \approx \min\left(A_v \cdot \frac{5\,|u|}{100} - V_{drop},\ V_{maks}\right), \qquad A_v = 1 + \frac{R_2}{R_1}, \tag{1}$$

dengan V_{drop} adalah rugi tegangan pada transistor dan L293D, dan V_{maks} adalah tegangan maksimum yang dapat dicapai rangkaian (sedikit di bawah 24 V). Bila penguatan A_v terlalu besar, tegangan maksimum sudah tercapai sebelum $u = 100\%$ sehingga rentang masukan di atasnya terbuang (saturasi dini). Bila A_v terlalu kecil, kecepatan maksimum motor tidak pernah tercapai. Penguatan yang baik membuat kecepatan maksimum tercapai tepat ketika masukan mendekati 100%. Penguatan diatur dengan trimpot R_2 pada rangkaian OpAmp.

2.3 Kompensasi *Deadzone*

Kompensasi *deadzone* bertujuan agar masukan kecil yang diberikan pengontrol langsung menghasilkan putaran. Caranya adalah memetakan masukan u ke masukan terkompensasi u_c yang melompati daerah *deadzone*. Bentuk paling sederhana adalah pemetaan linier

$$u_c = \begin{cases} DZ^+ + u\,\dfrac{100 - DZ^+}{100}, & u > 0, \\ 0, & u = 0, \\ -DZ^- + u\,\dfrac{100 - DZ^-}{100}, & u < 0, \end{cases} \tag{2}$$

dengan DZ^+ dan DZ^- adalah besar *deadzone* positif dan negatif dalam persen. Pemetaan ini menjaga $u_c = \pm 100$ ketika $u = \pm 100$. Persamaan (2) diimplementasikan pada tab `4_compensation.ino` melalui variabel `DeadZonePositif` dan `DeadZoneNegatif`. Pemilihan nilai DZ (misalnya dari masukan minimum putar atau masukan minimum terdeteksi) merupakan keputusan rancangan yang perlu dijustifikasi.

2.4 Model Orde Satu dengan Waktu Tunda

Dinamika kecepatan motor DC beserta sensornya dapat didekati dengan model FOPDT

$$G(s) = \frac{Y(s)}{U(s)} = \frac{K\,e^{-\theta s}}{\tau s + 1}, \tag{3}$$

dengan K adalah penguatan statik, τ adalah konstanta waktu, dan θ adalah waktu tunda. Untuk step masukan sebesar ΔU pada $t = 0$ dari keadaan tunak Y_0, respons model adalah

$$y(t) = \begin{cases} Y_0, & t < \theta, \\ Y_0 + K\,\Delta U\left(1 - e^{-(t-\theta)/\tau}\right), & t \geq \theta. \end{cases} \tag{4}$$

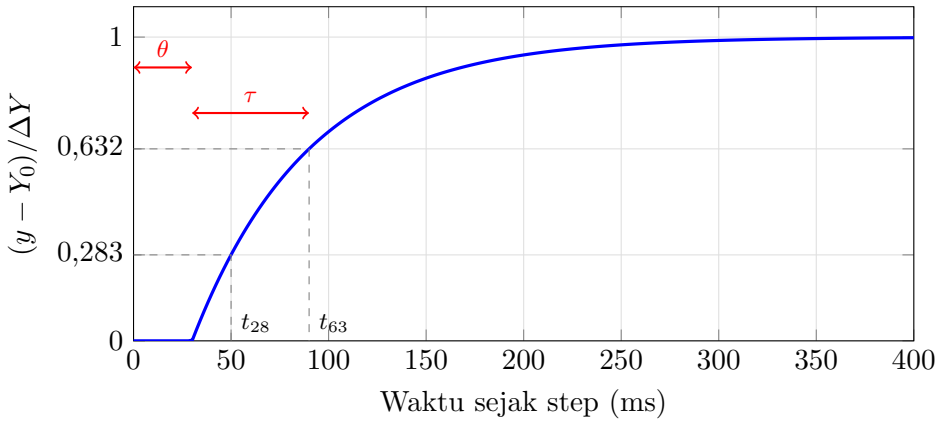
Konstanta waktu terutama ditentukan oleh inersia mekanik motor, sedangkan waktu tunda dapat berasal dari gesekan, waktu komputasi, pencacahan, dan rangkaian pengkondisi sinyal sensor.

2.5 Identifikasi Parameter dengan Metode Dua Titik

Parameter model FOPDT diperoleh dari respons step (Gambar 2):

1. **Penguatan**: $K = \Delta Y / \Delta U$, dengan $\Delta Y = Y_1 - Y_0$ adalah selisih keluaran tunak sesudah dan sebelum step, dan $\Delta U = U_1 - U_0$ adalah selisih masukan.
2. **Konstanta waktu dan waktu tunda** (metode dua titik [2]): tentukan waktu t_{28} dan t_{63} ketika respons mencapai 28,3% dan 63,2% dari ΔY, dihitung dari saat step diberikan. Dari Persamaan (4), kedua titik ini memenuhi $t_{28} = \theta + \tau/3$ dan $t_{63} = \theta + \tau$, sehingga

$$\tau = 1{,}5\,(t_{63} - t_{28}), \qquad \theta = t_{63} - \tau. \tag{5}$$



Gambar 2: Respons step ternormalisasi model FOPDT dan titik-titik yang digunakan pada metode dua titik ($\theta = 30$ ms, $\tau = 60$ ms)

Beberapa kesalahan yang sering terjadi dalam identifikasi:

- Menggunakan nilai absolut masukan dan keluaran, bukan selisihnya, pada step yang tidak dimulai dari nol. Akibatnya nilai K tidak sesuai.
- Menentukan saat step secara keliru sehingga waktu tunda bernilai negatif atau sangat kecil.
- Menahan masukan terlalu singkat sehingga keluaran belum mencapai keadaan tunak.

Karena sistem tidak sepenuhnya linier, parameter hasil identifikasi dapat berbeda untuk setiap besar masukan. Untuk keperluan perancangan, dipilih satu set parameter yang mewakili sistem, misalnya nilai rata-rata, median, atau nilai pada daerah kerja yang paling sering digunakan. Pilihan ini perlu dijustifikasi.

2.6 Pemilihan Waktu Cacah

Waktu cacah T_s harus memenuhi dua batasan:

1. **Batas bawah dari perangkat keras**: T_s harus lebih besar daripada waktu proses satu siklus program, $T_s > T_{proses}$, dengan margin yang cukup (lihat hasil Modul 0).
2. **Batas atas dari dinamika sistem**: T_s harus cukup kecil agar dinamika sistem terwakili. Aturan praktis yang umum digunakan adalah memilih 4–10 sampel selama waktu naik T_r [1]. Untuk sistem orde satu, $T_r \approx 2{,}2\,\tau$, sehingga

$$\frac{T_r}{10} \le T_s \le \frac{T_r}{4} \quad \Longrightarrow \quad 0{,}22\,\tau \lesssim T_s \lesssim 0{,}55\,\tau. \tag{6}$$

   Bila pengontrol dirancang secara kontinu lalu didiskritkan (seperti pada Modul 2), waktu cacah yang lebih kecil, misalnya $T_s \approx \tau/10$, akan membuat perilaku pengontrol diskrit lebih mendekati rancangan kontinunya.

Pada modul ini, praktikan menentukan **waktu cacah pertama** T_{s1} dan **waktu cacah kedua** $T_{s2} = 3\,T_{s1}$. Kedua waktu cacah ini akan dibandingkan pada Modul 2.

2.7 Model Diskrit

Bila masukan ditahan konstan selama satu waktu cacah oleh DAC (*zero-order hold*, ZOH) dan waktu tunda didekati sebagai kelipatan bulat waktu cacah, $\theta \approx d\,T_s$, model FOPDT (3) memiliki padanan diskrit

$$G(z) = \frac{b\,z^{-(d+1)}}{1 - a\,z^{-1}}, \qquad a = e^{-T_s/\tau}, \quad b = K\,(1-a), \quad d = \operatorname{round}\!\left(\frac{\theta}{T_s}\right), \tag{7}$$

atau dalam bentuk persamaan beda

$$y[k+1] = a\,y[k] + b\,u[k-d]. \tag{8}$$

Kutub sistem diskrit berada di $z = a$, yang selalu berada di dalam lingkaran satuan untuk $\tau > 0$. Penguatan tunak $G(1) = b/(1-a) = K$ tidak bergantung pada T_s. Model diskrit ini akan digunakan pada simulasi Modul 2 dan menjadi dasar perancangan *pole placement* pada Modul 4.

3 Keselamatan Kerja

Seluruh ketentuan keselamatan kerja dan SOP pada Modul 0 tetap berlaku. Perhatikan pula hal-hal berikut yang khusus untuk Modul 1:

1. Program `6_main_statik` dan `7_main_step` menjalankan motor secara otomatis selama beberapa menit. Jangan meninggalkan kit ketika program berjalan, dan cabut kabel listrik 220 V bila terjadi hal yang tidak diinginkan.
2. Untuk arah negatif pada `6_main_statik`, ubah `Sign` lalu unggah ulang program. Jangan membalik arah motor secara langsung.
3. Gunakan hanya masukan positif pada urutan `Sequence[]` di `7_main_step`.
4. Putar trimpot perlahan menggunakan obeng kecil. Jangan menyentuh komponen lain pada PCB, terutama *heat sink* transistor, ketika motor berputar.
5. Variabel `DeadZonePositif` dan `DeadZoneNegatif` bertipe `byte`, sehingga harus diisi bilangan bulat positif 0–100. Nilai *deadzone* negatif diisi besarnya saja.

4 Prosedur Praktikum

Prosedur Modul 1 terdiri atas enam percobaan. Percobaan 1–5 menggunakan kit praktikum, sedangkan Percobaan 6 sebagian besar dikerjakan di MATLAB dan dapat dilakukan secara paralel. Simpan seluruh data mentah dari *Serial Monitor* ke dalam file teks atau *spreadsheet*.

> **Catatan.** Data dari *Serial Monitor* dapat disalin langsung (pilih seluruh teks, salin, lalu tempel ke file `.txt`). Alternatifnya, gunakan perangkat lunak terminal serial yang dapat menyimpan log ke file. Pastikan *baudrate* 115200 dan pengaturan akhir baris **Newline**.

4.1 Percobaan 1: Karakteristik Sensor

Bagian A – sensor posisi. Kabel listrik 220 V tidak terhubung.

1. Unggah program utama `1_main_akuisisi_data` dari Modul 0.
2. Buka *Serial Plotter*, lalu putar roda gigi potensiometer dengan tangan secara perlahan sebanyak satu putaran penuh, termasuk melewati titik ketika bacaan berpindah dari sekitar 100% ke 0% (atau sebaliknya). Simpan tangkapan layar.
3. Biarkan potensiometer diam di sekitar 50%. Rekam minimal 100 baris data, lalu hitung rata-rata dan simpangan baku kolom posisi satu sampel (Pos 2) dan posisi rata-rata (Pos 3).

Bagian B – sensor kecepatan.

1. Unggah program utama `5_main_motor` dari Modul 0, lalu hubungkan kabel listrik 220 V.
2. Kirim masukan `75`. Setelah kecepatan tunak, rekam data selama sekitar 5 detik. Hitung rata-rata dan simpangan baku kolom kecepatan satu sampel (Kec 1) dan kecepatan rata-rata (Kec 2), serta rata-rata kolom `Delta`.
3. Kirim `0`. Ubah jumlah sampel perataan pada fungsi `smooth_velocity_percentage()` di tab `1_sensor.ino` (argumen kedua `smooth_pin`, bawaan 10) menjadi 1, 5, 20, dan 40. Untuk setiap nilai, ulangi langkah 2 dan lengkapi Tabel 2.
4. Kembalikan jumlah sampel perataan ke nilai yang menurut kelompok paling sesuai, lalu gunakan nilai tersebut untuk percobaan selanjutnya.

Catatan. Fungsi `smooth_pin()` menjumlahkan hasil `analogRead()` ke variabel bertipe `unsigned int` (maksimum 65535). Jangan gunakan jumlah sampel lebih dari 64.

Tabel 2: Pengaruh jumlah sampel perataan terhadap bacaan kecepatan (masukan 75%)

Jumlah sampel	Rata-rata Kec (%)	Simpangan baku Kec (%)	Rata-rata `Delta` (ms)	Keterangan
1				
5				
10				
20				
40				

4.2 Percobaan 2: Penyetelan Penguatan Driver Motor

1. Dengan program `5_main_motor`, kirim masukan `80`, `90`, `95`, dan `100` secara berurutan. Tunggu kecepatan tunak pada setiap masukan, lalu catat kecepatannya pada kolom “sebelum” Tabel 3.
2. Bila kecepatan pada masukan 90% atau 95% sudah sama dengan kecepatan pada 100%, penguatan terlalu besar. Dengan masukan `100`, putar trimpot

sedikit demi sedikit sambil mengamati *Serial Plotter* untuk menemukan arah putar yang menurunkan penguatan.

3. Turunkan penguatan perlahan hingga kecepatan pada masukan 100% **tepat mulai turun** dari nilai maksimumnya, lalu kembalikan sedikit hingga kecepatan maksimum tercapai lagi.
4. Kirim kembali masukan `80`, `90`, `95`, dan `100`, lalu catat kecepatannya pada kolom "sesudah". Kecepatan pada 95% seharusnya kini lebih kecil daripada kecepatan pada 100%.
5. Kirim `0`, lalu minta asisten memeriksa hasil penyetelan. **Jangan mengubah trimpot lagi** hingga akhir praktikum (termasuk Modul 2–4) karena model yang diidentifikasi bergantung pada penguatan ini.

Tabel 3: Kecepatan tunak sebelum dan sesudah penyetelan penguatan

Masukan (%)	**Kecepatan sebelum (%)**	**Kecepatan sesudah (%)**
80		
90		
95		
100		

4.3 Percobaan 3: Karakteristik Statik Tanpa Kompensasi

1. Ganti tab `9_main.ino` dengan program `6_main_statik` (Kode 1). Atur `StepInput = 1`, `Sign = 1`, `UseCompensation = false`, dan `PrintAll = false`.
2. Verifikasi program tanpa daya motor (kabel 220 V tidak terhubung) selama beberapa tingkat masukan, lalu unggah ulang program dan hubungkan kabel listrik. Program akan menaikkan masukan dari 0 hingga 100% dengan setiap tingkat ditahan 2 detik (sekitar 3,5 menit).
3. Setiap tingkat masukan menghasilkan satu baris: masukan, masukan motor, kecepatan tunak (rata-rata 1 detik terakhir), dan besar perubahan posisi selama tingkat tersebut. Salin seluruh hasil ke *spreadsheet*.
4. Cabut kabel listrik, ubah `Sign = -1`, unggah ulang, hubungkan kembali kabel listrik, lalu ulangi pengambilan data untuk arah negatif.
5. Buat grafik hubungan masukan dan kecepatan tunak untuk kedua arah dalam satu grafik.
6. Lengkapi Tabel 4. Gunakan kolom perubahan posisi untuk menentukan masukan minimum putar motor, dan kolom kecepatan tunak untuk menentukan masukan minimum terdeteksi. Gunakan simpangan baku dari Percobaan 1 sebagai acuan batas derau.
7. Bagi karakteristik statik menjadi beberapa daerah linier dan lengkapi Tabel 5.

Tabel 4: Spesifikasi kit praktikum berkaitan dengan hubungan masukan dan kecepatan (Nomor kit: _____)

No	Spesifikasi	Nilai	Satuan
1	Masukan minimum putar motor (positif)		%
2	Masukan minimum putar motor (negatif)		%
3	Masukan minimum terdeteksi (positif)		%
4	Masukan minimum terdeteksi (negatif)		%
5	Masukan minimum putaran maksimal (positif)		%
6	Masukan minimum putaran maksimal (negatif)		%
7	Kecepatan putar maksimum (positif)		%
8	Kecepatan putar maksimum (negatif)		%

Tabel 5: Pembagian daerah linier karakteristik statik

No	Rentang masukan (%)	Rentang keluaran (%)	m	R^2	Keterangan
1					
2					
3					
4					
5					

4.4 Percobaan 4: Kompensasi *Deadzone*

1. Berdasarkan Tabel 4, tentukan nilai DZ^+ dan DZ^-, lalu isi variabel `DeadZonePositif` dan `DeadZoneNegatif` pada tab `4_compensation.ino`. Bila diperlukan, kelompok boleh memodifikasi fungsi `compensation()` selama $u = 0$ tetap menghasilkan $u_c = 0$.
2. Pada `6_main_statik`, atur `UseCompensation = true`, `StepInput = 5`, dan `Sign = 1`. Verifikasi tanpa daya motor bahwa kolom `Motor` sesuai Persamaan (2), lalu ambil data karakteristik statik.
3. Ulangi untuk `Sign = -1`.
4. Buat grafik karakteristik statik dengan dan tanpa kompensasi dalam satu gambar, lalu lengkapi kembali tabel daerah linier (gunakan format Tabel 5) untuk sistem terkompensasi.
5. Bila *deadzone* belum berkurang setidaknya setengahnya, perbaiki nilai DZ atau fungsi kompensasi, lalu ulangi langkah 2–3.

4.5 Percobaan 5: Respons Step

1. Ganti tab `9_main.ino` dengan program `7_main_step` (Kode 2). Pastikan `UseCompensation = true` dan fungsi kompensasi dari Percobaan 4 sudah digunakan.
2. Atur `TimeSampling` sekecil mungkin tetapi tetap lebih besar daripada waktu proses (misalnya 10 ms). Periksa kolom `Delta` untuk memastikan waktu cacah konsisten.

3. **Data A – step dari nol**: gunakan urutan `{0, 20, 0, 40, 0, 60, 0, 80, 0, 100, 0}`. Verifikasi tanpa daya motor, lalu jalankan dan simpan seluruh data ke file `data_A.txt`.
4. Periksa apakah kecepatan sudah tunak sebelum masukan berikutnya. Bila belum, perbesar `HoldTime` dan ulangi.
5. **Data B – step berkelanjutan**: gunakan urutan `{0, 20, 40, 60, 80, 100, 0}`, lalu simpan ke file `data_B.txt`.

4.6 Percobaan 6: Identifikasi Model, Waktu Cacah, dan Model Diskrit

1. Buka skrip `identifikasi_fopdt.m` (Kode 3). Atur `NamaFile = 'data_A.txt'`, lalu jalankan. Skrip akan menampilkan parameter K, τ, dan θ untuk setiap step naik. Lengkapi Tabel 6.
2. Ulangi untuk `data_B.txt`.
3. Periksa hasil skrip dengan menghitung secara manual parameter salah satu step menggunakan *spreadsheet* dan Persamaan (5).
4. Tentukan satu set parameter K, τ, dan θ yang mewakili sistem, lalu tentukan T_{s1} berdasarkan Persamaan (6) dan waktu proses dari Modul 0. Tetapkan $T_{s2} = 3\,T_{s1}$. Lengkapi Tabel 7.
5. Isi parameter tersebut pada bagian parameter manual skrip, ubah variabel `GunakanParameterManual` menjadi `true`, dan isi nilai `Ts1`. Catat koefisien a, b, dan d model diskrit yang ditampilkan.
6. **Validasi**: atur `TimeSampling` pada `7_main_step` menjadi T_{s1}, ambil kembali data dengan urutan Data A, lalu jalankan skrip pada data baru tersebut. Catat nilai RMSE dan simpan grafik perbandingan. Ulangi untuk T_{s2}. Lengkapi Tabel 8.

Tabel 6: Hasil identifikasi parameter setiap step

Data	**Step**	ΔY (%)	K	τ **(ms)**	θ **(ms)**	**Ket.**
A	$0 \rightarrow 20$					
A	$0 \rightarrow 40$					
A	$0 \rightarrow 60$					
A	$0 \rightarrow 80$					
A	$0 \rightarrow 100$					
B	$0 \rightarrow 20$					
B	$20 \rightarrow 40$					
B	$40 \rightarrow 60$					
B	$60 \rightarrow 80$					
B	$80 \rightarrow 100$					

Tabel 7: Model final sistem kecepatan kit praktikum

No	Parameter	Nilai	Satuan	Model
1	Penguatan K		–	
2	Konstanta waktu τ		ms	
3	Waktu tunda θ		ms	
4	Kecepatan maksimum		%	
5	Waktu cacah pertama T_{s1}		ms	
6	a, b, d untuk T_{s1}		–	
7	Waktu cacah kedua T_{s2}		ms	
8	a, b, d untuk T_{s2}		–	

Tabel 8: Validasi model diskrit terhadap data percobaan

Waktu cacah	Nilai (ms)	RMSE (%)
T_{s1}		
T_{s2}		

5 Laporan dan Analisis

5.1 Ketentuan Laporan

Laporan dikumpulkan per kelompok melalui Edunex paling lambat dua minggu setelah praktikum. Nilai tambahan diberikan bila laporan dikumpulkan dalam satu minggu. Laporan berupa file `.pdf` yang dilampiri *spreadsheet* pengolahan data, data mentah (`.txt`), program Arduino yang dimodifikasi, dan skrip MATLAB, digabung dalam satu file `.zip`. Laporan minimal memiliki susunan berikut: latar belakang, tujuan, alat, prosedur, analisis, kesimpulan, kontribusi, dan referensi.

Latar belakang, tujuan, alat, dan prosedur tidak boleh sama persis dengan modul ini; tuliskan kembali sesuai pemahaman kelompok dan pelaksanaan yang sebenarnya. Analisis disusun dengan alur yang runtut dengan menjawab pertanyaan pada Bagian 5.2; urutan jawaban tidak harus sama dengan urutan pertanyaan. Perhatikan penggunaan angka penting pada setiap hasil perhitungan. Bagian kontribusi berisi Tabel 9.

Tabel 9: Kontribusi anggota kelompok

No	NIM	Nama Lengkap	Kontribusi
1			
2			
3			
4			
5			

5.2 Analisis

Karakteristik sensor.

1. Tunjukkan grafik bacaan potensiometer ketika diputar satu putaran penuh. Jelaskan mengapa bacaan berpindah langsung dari sekitar 100% ke 0% (atau sebaliknya). Apa konsekuensinya bagi kontrol posisi pada Modul 3?

2. Jelaskan kelebihan dan kekurangan akuisisi satu sampel (`sample`) dan perataan beberapa sampel (`smooth`) berdasarkan Tabel 2. Berapa jumlah sampel perataan yang kelompok pilih, dan apa pertimbangannya?
3. Mengapa jumlah sampel perataan tidak boleh lebih dari 64?
4. Apa saja yang memengaruhi waktu proses satu siklus program? Bagaimana cara mengoptimalkannya?

Penyetelan penguatan dan karakteristik statik.

5. Jelaskan bagaimana trimpot pada rangkaian OpAmp memengaruhi kecepatan maksimum motor berdasarkan Persamaan (1) dan Tabel 3. Mengapa penyetelan ini penting sebelum identifikasi?
6. Bacaan kecepatan selalu positif, baik untuk masukan positif maupun negatif. Mengapa demikian? Bagaimana tanda masukan memengaruhi arah putar motor?
7. Mengapa motor tidak berputar ketika masukan kecil? Mengapa bacaan kecepatan dapat bernilai nol walaupun motor sudah berputar pelan? Kaitkan dengan rangkaian pengkondisi sinyal tachometer pada Modul 0.
8. Tunjukkan grafik karakteristik statik tanpa kompensasi dan Tabel 4. Apakah karakteristik arah positif dan negatif simetris?
9. Jelaskan pembagian daerah linier yang dipilih beserta pertimbangannya (Tabel 5).

Kompensasi *deadzone*.

10. Gambarkan diagram blok kit praktikum untuk sistem kalang terbuka dan kalang terbuka terkompensasi. Jelaskan perbedaannya.
11. Jelaskan fungsi kompensasi yang digunakan dan tuliskan dalam persamaan matematika. Mengapa kelompok memilih nilai DZ^+ dan DZ^- tersebut? Apa akibatnya bila DZ dipilih terlalu kecil atau terlalu besar?
12. Tunjukkan grafik karakteristik statik dengan dan tanpa kompensasi dalam satu gambar. Apakah kompensasi berhasil mengurangi *deadzone*? Bandingkan pembagian daerah liniernya.

Identifikasi model.

13. Tunjukkan respons step Data A dan Data B beserta Tabel 6. Buat grafik parameter K, τ, dan θ terhadap besar masukan. Bagaimana parameter berubah terhadap titik kerja, dan apa artinya bagi asumsi linier?
14. Bandingkan hasil identifikasi step dari nol dan step berkelanjutan. Metode mana yang lebih sesuai untuk kit ini? Jelaskan.
15. Bandingkan hasil skrip dengan perhitungan manual. Apakah terdapat waktu tunda yang bernilai negatif atau sangat kecil? Jika ya, apa penyebabnya?
16. Pilih satu set parameter yang mewakili sistem dan jelaskan dasar pemilihannya.

17. Identifikasi secara manual konstanta waktu pada salah satu step **turun** (misalnya 100 → 0 pada Data A). Bandingkan dengan konstanta waktu step naik dan dengan konstanta waktu filter tachometer pada Modul 0 ($\tau_f = 0{,}1$ s). Apa yang dapat disimpulkan?

Waktu cacah dan model diskrit.

18. Jelaskan pertimbangan pemilihan T_{s1}. Apakah T_{s1} dan T_{s2} memenuhi Persamaan (6)?
19. Tuliskan model diskrit $G(z)$ untuk T_{s1} dan T_{s2}. Di mana letak kutubnya? Bagaimana letak kutub berubah bila waktu cacah diperbesar?
20. Tunjukkan grafik validasi dan Tabel 8. Apakah model diskrit cukup mewakili kit praktikum? Pada kondisi apa model paling berbeda dengan data, dan mengapa?

6 Tugas Tambahan

Tugas tambahan bersifat opsional dan dapat ditambahkan pada laporan untuk nilai tambah. Tugas ini juga merupakan persiapan Modul 2.

1. Hitung parameter pengontrol P, PI, dan PID menggunakan metode Ziegler–Nichols dan Cohen–Coon berdasarkan model final untuk kedua waktu cacah.
2. Simulasikan respons kalang terbuka dan kalang tertutup menggunakan program simulasi MATLAB yang disediakan. Pengontrol mana yang paling sesuai? Berikan justifikasi.
3. Identifikasi model orde dua dengan waktu tunda dari data yang sama, lalu bandingkan RMSE-nya dengan model FOPDT. Apakah peningkatan akurasinya sepadan dengan bertambahnya kompleksitas model?
4. Siapkan fungsi pada tab `5_control.ino` untuk menghitung aksi kontrol PID diskrit beserta program utama yang mengontrol kecepatan motor sesuai masukan serial.

Daftar Pustaka

[1] K. J. Åström dan B. Wittenmark, *Computer-Controlled Systems: Theory and Design*, edisi ke-3. Englewood Cliffs, NJ: Prentice Hall, 1997.

[2] D. E. Seborg, T. F. Edgar, D. A. Mellichamp, dan F. J. Doyle III, *Process Dynamics and Control*, edisi ke-4. Hoboken, NJ: Wiley, 2016.

A Kode Program Pendukung

Program utama berikut menggantikan isi tab `9_main.ino`. Tab lain (`1_sensor.ino` hingga `4_compensation.ino`) tetap sama seperti pada Modul 0.

Kode 1: Program utama `6_main_statik`

```
// ============================================================================
// Program utama Modul 1: karakteristik statik (masukan bertingkat)
// Ganti isi tab 9_main.ino dengan program ini.
//
// Masukan motor dinaikkan bertahap dari 0 hingga 100% (atau 0 hingga -100%
// bila Sign = -1). Setiap tingkat ditahan selama HoldTime. Kecepatan tunak
// dihitung sebagai rata-rata bacaan kecepatan pada AverageTime terakhir.
// Untuk arah negatif, ubah Sign menjadi -1 lalu unggah ulang program
// (jangan membalik arah motor secara langsung).
// ============================================================================

unsigned long Time, StartTime, LastTime, LevelStart;
unsigned int TimeSampling = 20, DeltaTime;   // waktu cacah (ms)
unsigned int HoldTime = 2000;                // lama tiap tingkat masukan (ms)
unsigned int AverageTime = 1000;             // rentang akhir tiap tingkat untuk
    rata-rata (ms)
int StepInput = 1;                           // kenaikan masukan tiap tingkat (%)
int Sign = 1;                                // 1: arah positif, -1: arah negatif
bool UseCompensation = false;                // true: masukan dikompensasi (
    Percobaan 4)
bool PrintAll = false;                       // true: cetak semua sampel, false:
    ringkasan tiap tingkat

int Level = 0;
float InputLevel = 0.0, MotorInput = 0.0, Motor = 0.0;
float Speed = 0.0, Position = 0.0, PositionStart = 0.0;
float SumSpeed = 0.0;
unsigned int CountSpeed = 0;
bool Finished = false;

void setup() {
  general_setup();
  sensor_setup();
  actuator_setup();
  print_header();
  StartTime = millis();
  LastTime = StartTime;
  LevelStart = StartTime;
  PositionStart = smooth_position_percentage();
}

void loop() {
  Time = millis();
  if (Time - LastTime >= TimeSampling) {
    DeltaTime = Time - LastTime;
    LastTime = Time;

    if (Finished) {
      run_motor(0.0);
      return;
    }

    InputLevel = (float) (Sign * Level * StepInput);
    if (UseCompensation) MotorInput = compensation(InputLevel);
```

```
    else MotorInput = InputLevel;
    Motor = run_motor(MotorInput);

    Speed = smooth_velocity_percentage();
    Position = smooth_position_percentage();

    if (Time - LevelStart >= (unsigned long) (HoldTime - AverageTime)) {
      SumSpeed += Speed;
      CountSpeed++;
    }

    if (PrintAll) print_sample();

    if (Time - LevelStart >= HoldTime) {
      if (!PrintAll) print_summary();
      SumSpeed = 0.0;
      CountSpeed = 0;
      LevelStart = Time;
      PositionStart = Position;
      Level++;
      if (Level * StepInput > 100) {
        Finished = true;
        Serial.println("# selesai");
      }
    }
  }
}

void print_header() {
  if (PrintAll) Serial.println("Time\tDelta\tInput\tMotor\tKec\tPos");
  else Serial.println("Input\tMotor\tKecTunak\tPerubahanPos");
}

void print_sample() {
  Serial.print(Time - StartTime);
  Serial.print("\t");
  Serial.print(DeltaTime);
  Serial.print("\t");
  Serial.print(InputLevel);
  Serial.print("\t");
  Serial.print(Motor);
  Serial.print("\t");
  Serial.print(Speed);
  Serial.print("\t");
  Serial.println(Position);
}

void print_summary() {
  float PositionChange = Position - PositionStart;
  if (PositionChange < 0) PositionChange = -PositionChange;
  Serial.print(InputLevel);
  Serial.print("\t");
  Serial.print(Motor);
  Serial.print("\t");
  if (CountSpeed > 0) Serial.print(SumSpeed / CountSpeed);
  else Serial.print(Speed);
  Serial.print("\t");
  Serial.println(PositionChange);
}
```

Kode 2: Program utama `7_main_step`

```
// ============================================================================
// Program utama Modul 1: respons step untuk identifikasi dinamika
// Ganti isi tab 9_main.ino dengan program ini.
//
// Masukan motor mengikuti urutan Sequence[]; setiap nilai ditahan selama
// HoldTime. Data waktu, masukan, dan kecepatan dicetak setiap waktu cacah
// sehingga dapat disalin ke spreadsheet dan diolah dengan MATLAB.
//
// Contoh urutan:
//   step dari nol      : {0, 20, 0, 40, 0, 60, 0, 80, 0, 100, 0}
//   step berkelanjutan : {0, 20, 40, 60, 80, 100, 0}
// Gunakan masukan positif saja agar arah motor tidak berbalik.
// ============================================================================

unsigned long Time, StartTime, LastTime, StepStart;
unsigned int TimeSampling = 10, DeltaTime;   // waktu cacah (ms)
unsigned int HoldTime = 2000;                // lama tiap nilai masukan (ms)
bool UseCompensation = true;                 // gunakan kompensasi deadzone dari
    Percobaan 4

int Sequence[] = {0, 20, 0, 40, 0, 60, 0, 80, 0, 100, 0};
const byte SequenceLength = sizeof(Sequence) / sizeof(Sequence[0]);

byte Index = 0;
float InputStep = 0.0, MotorInput = 0.0, Motor = 0.0, Speed = 0.0;
bool Finished = false;

void setup() {
  general_setup();
  sensor_setup();
  actuator_setup();
  Serial.println("Time\tDelta\tInput\tMotor\tKec");
  StartTime = millis();
  LastTime = StartTime;
  StepStart = StartTime;
}

void loop() {
  Time = millis();
  if (Time - LastTime >= TimeSampling) {
    DeltaTime = Time - LastTime;
    LastTime = Time;

    if (Finished) {
      run_motor(0.0);
      return;
    }

    InputStep = (float) Sequence[Index];
    if (UseCompensation) MotorInput = compensation(InputStep);
    else MotorInput = InputStep;
    Motor = run_motor(MotorInput);
    Speed = smooth_velocity_percentage();

    Serial.print(Time - StartTime);
    Serial.print("\t");
    Serial.print(DeltaTime);
    Serial.print("\t");
    Serial.print(InputStep);
```

```
    Serial.print("\t");
    Serial.print(Motor);
    Serial.print("\t");
    Serial.println(Speed);

    if (Time - StepStart >= HoldTime) {
      StepStart = Time;
      Index++;
      if (Index >= SequenceLength) {
        Finished = true;
        Serial.println("# selesai");
      }
    }
  }
}
```

Kode 3: Skrip MATLAB `identifikasi_fopdt.m`

```
%% identifikasi_fopdt.m
% Identifikasi model orde satu dengan waktu tunda (FOPDT) dari data respons
% step kit praktikum Sistem Kontrol Diskrit (Modul 1).
%
% Data berasal dari program utama 7_main_step yang disalin dari Serial
% Monitor ke file teks (.txt) atau .csv dengan kolom:
%   Time (ms) | Delta (ms) | Input (%) | Motor (%) | Kec (%)
%
% Skrip ini:
%   1. mencari setiap step masukan naik (Delta U > 0),
%   2. menghitung K, tau, dan theta setiap step dengan metode dua titik
%      (28,3% dan 63,2%),
%   3. menghitung model diskrit (ZOH) untuk dua waktu cacah,
%   4. membandingkan respons model diskrit dengan data.

clc; clear; close all;

%% 1. Pengaturan
NamaFile      = 'data_step.txt';  % nama file data
KolomWaktu    = 1;                % kolom waktu (ms)
KolomMasukan  = 3;                % kolom masukan (%)
KolomKeluaran = 5;                % kolom kecepatan (%)
FraksiTunak   = 0.3;              % porsi akhir tiap segmen untuk nilai tunak
LebarFilter   = 3;                % lebar rata-rata bergerak untuk mencari titik
    lintas

Ts1 = 10;                         % waktu cacah pertama (ms), ubah sesuai pilihan
    kelompok
Ts2 = 3*Ts1;                      % waktu cacah kedua (ms)

% Parameter manual: isi dengan model final kelompok untuk memvalidasi model
% pada data baru (Percobaan 6). Bila false, digunakan median hasil step.
GunakanParameterManual = false;
K_manual     = 0.9;               % (-)
Tau_manual   = 60;                % (ms)
Theta_manual = 30;                % (ms)

%% 2. Membaca data
Data = readmatrix(NamaFile);
Kolom = [KolomWaktu KolomMasukan KolomKeluaran];
Data = Data(all(~isnan(Data(:, Kolom)), 2), :);   % buang baris teks / kosong
t = Data(:, KolomWaktu);
u = Data(:, KolomMasukan);
```

```
y = Data(:, KolomKeluaran);
t = t - t(1);
N = numel(t);

%% 3. Identifikasi parameter setiap step naik
IndeksStep = find(diff(u) ~= 0) + 1;     % sampel pertama setelah masukan berubah
Batas = [1; IndeksStep; N + 1];          % segmen ke-j: Batas(j) .. Batas(j+1)-1

Hasil = [];
for i = 1:numel(IndeksStep)
    SegSebelum = Batas(i):Batas(i+1)-1;
    SegSesudah = Batas(i+1):Batas(i+2)-1;
    U0 = u(SegSebelum(1));
    U1 = u(SegSesudah(1));
    dU = U1 - U0;
    if dU <= 0
        continue;                        % hanya step naik yang diidentifikasi
    end

    Y0 = mean(y(SegSebelum(end - porsi(numel(SegSebelum), FraksiTunak) + 1:end)));
    Y1 = mean(y(SegSesudah(end - porsi(numel(SegSesudah), FraksiTunak) + 1:end)));
    dY = Y1 - Y0;
    K  = dY / dU;

    tt = t(SegSesudah) - t(SegSesudah(1));
    r  = (movmean(y(SegSesudah), LebarFilter) - Y0) / dY;   % respons ternormalisasi
    t28 = waktu_lintas(tt, r, 0.283);
    t63 = waktu_lintas(tt, r, 0.632);
    Tau   = 1.5 * (t63 - t28);
    Theta = t63 - Tau;
    if Theta < 0
        warning('Step %g -> %g: waktu tunda negatif (%.1f ms), periksa data.', U0,
    U1, Theta);
    end

    Hasil(end+1, :) = [U0 U1 dU Y0 Y1 dY K Tau Theta]; %#ok<SAGROW>
end

if isempty(Hasil)
    error('Tidak ditemukan step masukan naik pada data.');
end

TabelHasil = array2table(Hasil, 'VariableNames', ...
    {'U0','U1','dU','Y0','Y1','dY','K','Tau_ms','Theta_ms'});
disp('Hasil identifikasi setiap step:');
disp(TabelHasil);

%% 4. Nilai yang mewakili sistem
% Secara bawaan digunakan median. Ganti dengan nilai pilihan kelompok bila
% ada alasan yang lebih kuat (misalnya hanya memakai daerah linier).
if GunakanParameterManual
    K     = K_manual;
    Tau   = Tau_manual;
    Theta = Theta_manual;
else
    K     = median(Hasil(:, 7));
    Tau   = median(Hasil(:, 8));
    Theta = max(median(Hasil(:, 9)), 0);
end
fprintf('\nModel kontinu: G(s) = %.3f e^(-%.1f s) / (%.1f s + 1)   [s dalam ms]\n',
    ...
```

```
    K, Theta, Tau);

%% 5. Model diskrit (ZOH) untuk dua waktu cacah
fprintf('\nModel diskrit y[k+1] = a y[k] + b u[k-d]:\n');
for Ts = [Ts1 Ts2]
    a = exp(-Ts / Tau);
    b = K * (1 - a);
    d = round(Theta / Ts);
    fprintf('  Ts = %3g ms : a = %.4f, b = %.4f, d = %d  ->  G(z) = %.4f z^-%d / (1 - %.4f z^-1)\n', ...
        Ts, a, b, d, b, d + 1, a);
end

%% 6. Validasi: simulasi model diskrit pada waktu cacah data
Tdata = median(diff(t));
a = exp(-Tdata / Tau);
b = K * (1 - a);
d = round(Theta / Tdata);
ym = zeros(N, 1);
ym(1) = y(1);
for k = 1:N-1
    if k - d >= 1
        uk = u(k - d);
    else
        uk = u(1);
    end
    ym(k + 1) = a * ym(k) + b * uk;
end
RMSE = sqrt(mean((y - ym).^2));
fprintf('\nValidasi pada Ts data = %g ms: RMSE = %.2f %%\n', Tdata, RMSE);

figure;
stairs(t, u, 'k-.'); hold on;
plot(t, y, 'b-');
stairs(t, ym, 'r-', 'LineWidth', 1.2);
grid on;
xlabel('Waktu (ms)'); ylabel('Persen');
legend('Masukan', 'Kecepatan terukur', 'Model diskrit', 'Location', 'best');
title(sprintf('K = %.3f, \\tau = %.1f ms, \\theta = %.1f ms, RMSE = %.2f%%', K, Tau,
     Theta, RMSE));

%% Fungsi bantu
function n = porsi(Panjang, Fraksi)
    n = max(1, round(Fraksi * Panjang));
end

function tc = waktu_lintas(tt, r, Level)
    % waktu pertama respons ternormalisasi r melintasi Level (interpolasi linier)
    k = find(r >= Level, 1, 'first');
    if isempty(k)
        tc = NaN;
    elseif k == 1
        tc = tt(1);
    else
        tc = tt(k-1) + (Level - r(k-1)) * (tt(k) - tt(k-1)) / (r(k) - r(k-1));
    end
end
```